

Switch, Reason, and Revise: Enhancing Reasoning Capability of Video Game AI by Large Language Models

Wei Li¹, Hongming Liu¹, Jiali Lv¹, Kaizhu Huang¹, *Senior, IEEE*, Aiguo Song¹, *Senior, IEEE*, and Zhen Lei², *Fellow, IEEE*

Abstract—Attributing to the strong reasoning capability, behavior models in artificial intelligence for games play a crucial role in creating gaming experiences. For further enhancing the reasoning capability of behavior models, we propose a novel Switch, Reason, and Revise (SRR) framework, which integrates them with Large Language Model (LLM). The SRR framework contains three core components. The component of Dual-Track Experiential Reasoning fully utilizes the agent experiences for reasoning. The component of Block-Retrieval-Augmented Thoughts adaptively determines the granularity of information retrieval for external sources. The component of Self-Reliant Thinking System Switch increases the reasoning speed by model switching and performs the model switching automatically upon the LLM. Together, these three components can strengthen the agent reasoning capability in complex tasks. Experimental results in the Pokémon battle environment demonstrate the effectiveness and efficiency superiority of SRR over the rival methods. Furthermore, we conduct an exploratory study to reveal the potential of the SRR-empowered agent for guiding new players in Pokémon battle games.

Index Terms—Artificial Intelligence for Games, Large Language Models, Dual-Track Experiential Reasoning, Block-Retrieval-Augmented Thoughts, Self-Reliant Thinking System Switch.

I. INTRODUCTION

Artificial Intelligence (AI) in video games typically refers to a system capable of observing its surroundings and taking actions to achieve goals. Common forms of game AI include both player and non-player characters, which can be collectively referred to as ‘bots’ [1]. Sophisticated bots are the key factors to induce the immersive experiences of players in a game. Behavior models, which directly control the actions and responses of bots, play an important role in game AI [2]. Common behavior models include rule-driven systems [3], [4] and learning-based approaches [5]–[9].

Recent breakthroughs in Large Language Model (LLM), achieved by large-scale pretraining on diverse textual corpora,

have endowed behavior models with strong reasoning capability. Reasoning capability refers to the ability of an agent to select the actions from its executable action set that is valuable to task completion. This capability has catalyzed a paradigm shift in game AI, for which researchers are now attempting to deploy LLM-based agents across a variety of video games [10]–[14]. Meanwhile, three characteristics of modern video games pose challenges to the application of LLM-based agents.

The first characteristic of modern video games is that their extended gameplay sessions necessitate the behavior models to implement behavioral strategy adaptation [15]. It leads to the first challenge: how can an LLM-based agent improve its behavioral strategy in games? Some In-Context Learning (ICL) methods are proposed to tackle this challenge [16]–[19]. However, current experience-based methods generate and aggregate the experiences without distinguishing between the distinct experiential sorts. Such limitation gives rise to our first research question: *can guiding the LLM-based agent to reflect from multiple perspectives benefit its behavioral strategy?*

To overcome the shortcoming of the existing methods which do not distinguish experiences, we design a component, Dual-Track Experiential Reasoning. This component guides the LLM-based agent to reflect from three perspectives, thus yielding three different but complementary sorts of experiences: the concrete experiences acquired from task trajectories, the abstract experiences summarized from strategic patterns, and the action outcome predictions integrating both concrete and abstract experiences.

As the second characteristic of modern video games, it is necessary for the behavior models to master intense domain knowledge before successfully completing the game tasks [20]. It leads to the second challenge: how can the LLM-based agent acquire the game-relevant information to support its reasoning? This challenge is inherently tied to the hallucination problem in the LLM, which generate the plausible but factually incorrect contents when the reliable external sources like gaming rule databases are deficient [21]. Current retrieval-augmented methods mitigate the hallucination problem, but they suffer from the logical incoherence during multi-step reasoning due to the inappropriate retrieval granularity [22]–[25]. Such incoherence raises our second research question: *can the LLM-based agent leverage its text analysis ability to*

Wei Li, Hongming Liu, Jiali Lv, and Aiguo Song are with the School of Instrument Science and Engineering, Southeast University, Nanjing, Jiangsu 210096, China. Kaizhu Huang is with the Digital Innovation Research Center, Duke Kunshan University, Kunshan, Jiangsu 215316, China. Zhen Lei is with the Institute of Automation, Chinese Academy of Sciences, Beijing 100190, China. *Corresponding author: Wei Li (li-wei@seu.edu.cn).*

This work was supported in part by the Open Fund of Key Laboratory of the Ministry of Education on Artificial Intelligence in Equipment under Grant 2024-AAIE-KF03-01 and in part by the State Key Laboratory of Multimedia Information Processing Open Fund under Grant SKLMIP-KF-2025-02.

0000–0000/00\$00.00 © 2021 IEEE

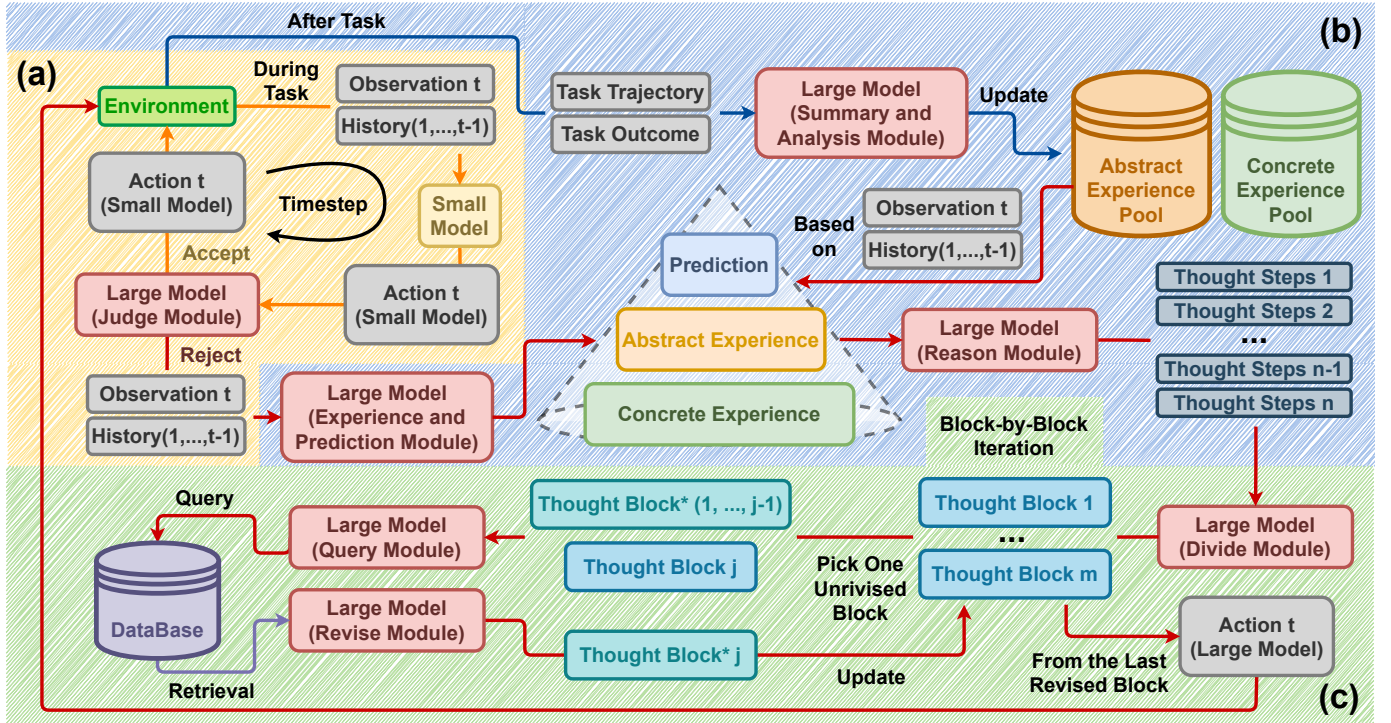


Fig. 1. Illustration of the SRR framework. (a) Self-Reliant Thinking System Switch. At each timestep, the small model makes an initial decision, and then the large model assesses the reasonability of this decision. If the decision is reasonable, the agent will execute it; otherwise, the large model intervenes to make a new decision, and jumps to the processes outlined in (b) and (c). (b) Dual-Track Experiential Reasoning. In a task, the large model selects relevant experiences from the two experience pools, and makes predictions based on the current observation and historical trajectories of agent. By utilizing these experiences and predictions, the large model engages in reasoning to generate a thought chain consisting of several thought steps. After the task, the large model updates the experience pools based on the task trajectories and outcomes. (c) B-RAT. The thought chain is segmented into several thought blocks and revised in three steps: generating query, information retrieval, and revision. And the fully revised thought chain contains the final reasoning outcome.

autonomously determine the retrieval granularity for maintaining the textual logical coherence?

To cope with the incoherence caused by the independent-step retrieval granularity, we devise a component, Block-Retrieval-Augmented Thoughts (B-RAT). This component employs the text analysis ability of LLM to dynamically segment the thought chain into several logically-coherent thought blocks. By utilizing the thought blocks as the retrieval granularity, B-RAT can reduce the logical contradictions when generating the query for the external sources, thus behaving better in information retrieval to support the inference revision.

The third characteristic of modern video games is that their interactive manner imposes the temporal boundaries on the decision-making process of behavior model [26]. It leads to the third challenge: how can the LLM-based agent reduce its decision latency? Existing methods try to reduce the engagement of LLM in reasoning [27], [28]. However, the predefined rules for model switching impede these methods from adapting to the dynamic environment more or less. This limitation motivates our third research question: *can the LLM-based agent achieve autonomous model switching based on the intrinsic reasoning capability, without using any predefined rule?*

To reduce the dependency on the predefined switching rules, we develop a component, Self-Reliant Thinking System Switch. This component replaces the predefined rules with LLM-driven model switching. Procedurally, at each decision

step, System 1 first generates an immediate action to prioritize the fast responses, and then System 2 assesses the reasonableness of this action. If System 2 has verified the feasibility of this action, the action will be executed; otherwise, System 2 will activate its reasoning module to regenerate an alternative action.

We propose to integrate the above three components into the Switch, Reason, and Revise (SRR) framework, as illustrated in Fig. 1. During the experimental validation of the SRR framework via the Pokémon battle game, we have observed that many novice players of this game rely on self-study or tutorials to master the essential gameplay skills. This phenomenon causes our fourth exploratory question: *can the LLM-based agent with the enhanced reasoning capacity serve as a mentor for novice players?*

To probe into this question, we recruit several volunteers to join in our comparative experiments. We use the LLM-based agent to guide the novice players in learning how to play the Pokémon battle game, and compare them not only with the players who learn from document tutorials and video tutorials but also with the players who completely explore on their own.

On the whole, our contributions are summarized as follows.

- (a) In SRR, we design the component of Dual-Track Experiential Reasoning to overcome the limitation of current ICL methods that focus on single-perspective reflection. With Dual-Track Experiential Reasoning, SRR can guide the LLM-based agent to reflect from three perspectives,

thus largely enhancing the behavioral strategy.

- (b) In SRR, we devise the component of B-RAT to overcome the limitation of current methods for which the independent thought-step retrieval causes the logical inconsistency in texts. By leveraging the semantically-coherent thought blocks as the retrieval units, B-RAT can tackle the context conflict during external source querying, thus enhancing the effectiveness of reasoning revision.
- (c) In SRR, we develop the component of Self-Reliant Thinking System Switch to substitute for the heuristic switching strategy, which heavily relies on manual effort, to suit the dynamic environment.

On the basis of SRR, our findings from the experiments are also provided as below. Experimental results in the Pokémon battle environment not only have validated that SRR can substantially improve the performance of agent when it is equipped with different LLMs, but also have verified that the SRR-empowered agent can serve as an instructive mentor for novice players in the Pokémon battle games.

II. RELATED WORK

A. In-Context Learning

ICL is a typical technique to improve the reasoning capability of LLM-based agent. ICL utilizes a set of demonstrations as part of the prompt, and combines them with the input texts of LLM. For instance, Chain-of-Thought (CoT) [29], ReAct [30] and HuggingGPT [31] utilize a divide-and-conquer approach to segment the complex tasks into relatively simple sub-tasks. Tree-of-Thought (ToT) [32], Graph-of-Thought (GoT) [33] and Self-Consistency (SC) [19] generate the multiple plans by LLMs. Besides, Reflexion [16], Self-Correcting with Tool-Interactive Critiquing (CRITIC) [17] and Self-Refine [18] incorporate the agent reflection on the past tasks into demonstrations. In these methods, the reward fed back from the environment is converted into the reflection in the natural language form. Such reflection is referred to as ‘experience’.

Nevertheless, current ICL methods only rely on one type of experiences to generate the reflections, indicating that these methods merely reflect from single one perspective. This method paradigm inevitably limits the effectiveness of demonstrations. In SRR, our designed Dual-Track Experiential Reasoning can handle such weakness. Dual-Track Experiential Reasoning enhances the reasoning capability of LLM-based agent by guiding the agent to reflect from three perspectives: the concrete experiences (i.e. the specific operation details), the abstract experiences (i.e. the reusable strategies), and the predicted action outcomes.

B. Retrieval-Augmented Generation

RAG can cope with the hallucination problem of LLM by integrating reliable information into the prompt, thus enhancing the reasoning capability of agent [34]. To improve the native RAG, the pre-retrieval optimization methods, such as Rewrite-Retrieve-Read [22], Bridges the Semantic Gap for Long-Tail Queries [35], Take-a-Step-Back [23] and Hypothetical Document Embeddings [24], improve the indexing structures of retrieval by means of query rewriting, transformation,

expansion, etc. And for the same purpose, the embedding representation approaches, like Angle-Optimized Embedding [36] and Voyager [37], enhance the retrieval performance by refining the semantic representation of text embeddings. Note that it is challenging to convert the whole thought chain into one single query when RAG is employed for CoT. This will inevitably result in the poor performance of agent, especially in the multi-step reasoning scenarios. To cope with this problem, Retrieval-Augmented Thoughts (RAT) [25] retrieves the information tailored to every intermediate thought step for revising the thought chain.

Nevertheless, the granularity of information retrieval based on the independent thought steps is easy to incur the crucial problems in the textual logical coherence during information retrieval and reasoning revision. Specifically, in the course of multi-step reasoning, this kind of information retrieval technique may induce the inter-step logical contradiction. Such contradiction will bring about the inappropriate query for the external sources, consequently resulting in the retrieved information which mismatches the adjacent contexts. In SRR, our devised B-RAT can deal with this problem. B-RAT leverages the LLM to partition the thought chain into semantically-coherent thought blocks, which can serve as the context-aware operational units for information retrieval. Such block-level granularity can preserve the internal logical coherence while facilitating the context integration during information retrieval.

C. Dual-Process Theory

Dual-Process Theory [38] offers a rational psychological framework explaining the reasoning process of two cognitive systems. System 1 is intuitive, heuristic, and works quickly, while System 2 is deliberate, logical, and requires conscious effort and attention. Inspired by this theory, both SwiftSage [27] and LLM-Planning-Viz [28] suggest using a small model as System 1 (fast intuition) and an LLM as System 2 (slow reasoning). To reduce the decision-making delay, SwiftSage and LLM-Planning-Viz resort to heuristic algorithms to switch the two systems. During switching, a subset of tasks is delegated to System 1 rather than System 2.

Even so, the heuristic switching algorithm heavily relies on the predefined rules, which not only require laborious manual efforts but also adapt tardily to the dynamic environment. In SRR, our developed Self-Reliant Thinking System Switch can tackle with this limitation. In Self-Reliant Thinking System Switch, the LLM autonomously assesses the reasonableness of decisions from System 1 and determines whether to switch to System 2 for further reasoning.

III. METHOD

A. Background

1) *In-Context Learning*: Given input text x , a set of m candidate answers $Y = \{y_i\}_{i=1}^m$, and a set of k demonstrations $C = \{s(x_i, y_i, I_i)\}_{i=1}^k$, where each y_i denotes a token sequence, $s(x_i, y_i, I_i)$ denotes a demonstration of input-output pair with a task instruction, and I_i indicates the instruction of

this task, the probability $P(y_i|x)$ of the candidate answer y_i is

$$P(y_i|x, C) = f_{\mathcal{M}}(y_i, x, C), \quad (1)$$

where $f_{\mathcal{M}}$ presents the generation function based on the LLM denoted as $\mathcal{M}$. Thus, the selected answer $\hat{y} \sim P(y_i|x, C)$ [39].

We expect LLM to learn the contextual patterns hidden behind the provided demonstrations, rather than updating its parameters. Specifically, ICL should operate within a hypothesis class $\mathcal{H} = \{h(\cdot; \theta(C)) | \theta(C)\}$ [40]. Here, $\theta(C)$, which encodes C into the task-relevant features, is produced by the shallow layers of Transformer of the LLM. Based on the task vector, we can construct a new function $h(y_i, x; \theta(C))$, which is analogous to $f_{\mathcal{M}}(y_i, x, C)$ [40].

From the angle of parameter estimation, the generation process of task vector θ can be formalized as an implicit maximum likelihood estimation. The optimization objective of ICL is to minimize the divergence between the task distribution and the parameterized conditional distribution:

$$\hat{\theta}(C) = \arg \max_{\theta} \sum_{(x_i, y_i) \in C} \log h(y_i|x; \theta(C)). \quad (2)$$

ICL implicitly computes $\theta(C)$ by the shallow Transformer layers. By doing so, ICL can compress the task-relevant conditional distribution information into a single vector. $\theta(C)$ has been validated to be equivalent with a sufficient statistic of the task parameter distribution [40].

The efficacy of $\theta(C)$ depends not only on the pre-trained mapping function $\theta(\cdot)$, but also on the content and organization of the demonstrations within C . CoT [29] can enhance the reasoning capability of LLM by embedding the structured demonstrations into the prompt. But they lack the ability to continuously improve the behavioral strategy of LLM-based agent, because their demonstrations C remain unchanged and the task vector $\theta(C)$ is fixed.

Reflexion [16] takes advantage of the reflections to enrich the demonstrations C and improve the task vector $\theta(C)$. In subsequent tasks, Reflexion makes use of the experiences $E = \{M_e(\tau_i, r_i)\}_{i=1}^k$ to substitute for the demonstrations C , where r denotes the reward, τ denotes the trajectory, and M_e represents the LLM responsible for producing the environmental feedback. Therefore, the probability of the candidate answer y_i is

$$P(y_i|x) = h(y_i, x; \theta(E)), \quad (3)$$

where $\theta(E)$ denotes the task vector which describes the task-relevant features encoded from the experiences E . Since E is derived from agent-environment interactions, it is inherently task-relevant and can dynamically grow in the progress of task. So, compared with $\theta(C)$, the task vector $\theta(E)$ can more effectively deliver the task-relevant information into the reasoning process of LLM [16].

2) *Retrieval-Augmented Generation*: Given a set of n external documents $D = \{d_i\}_{i=1}^n$, RAG aims to retrieve the most relevant ones with respect to the query q . RAG first employs the text embedding model $\mathcal{E}$ to acquire the semantic embeddings $\{f_{\mathcal{E}}(d_i)\}_{i=1}^n$ and $f_{\mathcal{E}}(q)$ from both external documents

and query input, and then utilizes the cosine similarity $\text{sim}(\cdot, \cdot)$ between the semantic embeddings of every external document d_i and each query input q to measure their relevance:

$$\text{sim}(d_i, q) = \frac{f_{\mathcal{E}}(d_i) \cdot f_{\mathcal{E}}(q)}{\|f_{\mathcal{E}}(d_i)\| \|f_{\mathcal{E}}(q)\|}, \quad (4)$$

where $\|\cdot\|$ represents the L^2 -norm. Usually, the documents with the top- k similarity scores are prioritized for retrieval and being integrated into the prompt of LLM.

CoT, which can be characterized by multi-step reasoning, is competent in dealing with the complex tasks. The thought chain produced by CoT has been formulated by

$$T = f_{\mathcal{M}}(Y, x, I_c), \quad (5)$$

where $T = \{t_i\}_{i=1}^n$ denotes the thought chain comprising n thought step t , and I_c denotes the instruction that prompts the agent to produce the thought chain. T will then be added to the demonstrations to yield the final answer. RAT is proposed to revise every intermediate thought step in the thought chain formed by CoT. Given the thought chain comprising n thought steps, RAT performs n iterations of retrieval and revision. In the i -th iteration, RAT combines the sequence of revised intermediate thought steps $T_{1:i-1}^* = \{t_j^*\}_{j=1}^{i-1}$, the current thought step t_i , and the instruction I_q together for generating q_i :

$$q_i = f_{\mathcal{M}}(Y, x, T_{1:i-1}^*, t_i, I_q). \quad (6)$$

RAT carries out information retrieval based on the semantic embeddings $f_{\mathcal{E}}(q_i)$, so as to obtain the top- k relevant documents $D_i = \{d_j\}_{j=1}^k$. These documents are subsequently encapsulated into the prompt of LLM to generate the revised thought steps $T_{1:i}^*$:

$$T_{1:i}^* = f_{\mathcal{M}}(Y, x, D_i, I_r), \quad (7)$$

where I_r indicates the instruction that prompts the agent to revise the thought steps. After n iterations, $T^* = T_{1:n}^*$. At last, T^* replaces T for RAT to produce the final answer.

It is worth mentioning that, when utilizing individual thought steps as the retrieval granularity, there could be textual logical incoherence between the revised thought steps $T_{1:i-1}^*$ and the unrevised thought step t_i . For instance, although a causal correlation may exist between unrevised $T_{1:i-1}$ and t_i , the causality can be disrupted after revisions, such that $T_{1:i-1}^*$ and t_i become uncorrelated. From another angle, we are able to interpret the process of generating q_i as ICL, which is formulated as $q_i = h(Y, x; \theta(\hat{C}))$, where $\hat{C} = \{T_{1:i-1}^*, t_i, I_q\}$. When $T_{1:i-1}^*$ and t_i conflict with each other, the quality of task vector $\theta(\hat{C})$ will degrade, thereby resulting in the suboptimal query q_i . Consequently, the effectiveness of both information retrieval and revision will be reduced.

B. Problem Formulation

When the LLM-based agent operates as a game AI bot, the task can be formalized as a Partially Observable Markov Decision Process (POMDP), denoted as $M\langle S, A, P, R, \Omega, O, \gamma \rangle$. S represents the set of all possible environmental states, while A denotes the set of executable actions by the agent. The

state transition probability P is defined as the conditional probability $P(s_{i+1}|s_i, a_i)$, where $s_i \in S$ is the current state, $s_{i+1} \in S$ is the next state, and $a_i \in A$ is the action of agent. The observation space Ω encompasses all perceptible observations for the agent, and the observation probability O characterizes the likelihood of obtaining the observation $o_{i+1} \in \Omega$ after executing action a_i , defined as $O(o_{i+1}|s_i, a_i)$. The design of the reward function R varies across environments: in some scenarios, $R(s_i, a_i)$ represents the immediate reward for performing action a_i in state s_i , with cumulative rewards discounted by the factor $\gamma \in [0, 1]$; in others, the reward depends solely on the terminal state s_{final} , denoted as $R(s_{\text{final}})$. The objective of agent is to maximize the final reward by effective interaction with the environment.

C. Assigning Roles to LLM in Prompt-Based Inference

Recent studies have discovered that adding descriptions of the role to LLM prompt can help the LLM-based agent complete tasks more effectively [41]. In the SRR framework, LLM assumes two distinct roles: one dedicated to decision-making and selecting appropriate actions, and the other focused on evaluating decision quality, revising decision process, and reflecting on decision outcomes. In the flowchart of SRR framework, the two roles of LLM are represented by the Pokémon Player and Pokémon Coach in Pokémon battle environment, as illustrated in Fig. 2.

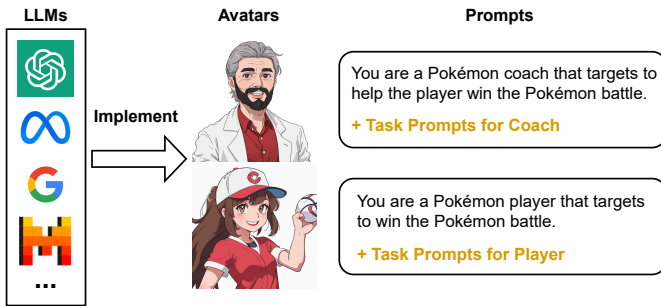


Fig. 2. The avatars of two roles of LLM in the flowchart of SRR. The Pokémon Player represents the LLM responsible for decision-making and action selection, while the Pokémon Coach corresponds to the LLM that evaluates decision quality, revises the decision process, and reflects on decision outcomes.

D. Dual-Track Experiential Reasoning

In Section III-A1, we have formalized the generation process of task vector θ as the implicit maximum likelihood estimation. The covariance matrix of parameter estimation $\hat{\theta}$ can be formulated by the Cramer-Rao lower bound as $\text{Cov}(\hat{\theta}) \succeq \mathcal{I}(\theta)^{-1}$ [42]. Here, $\text{Cov}(\hat{\theta})$ is the covariance matrix and $\mathcal{I}(\theta)$ is the Fisher information matrix, defined by

$$\mathcal{I}(\theta) = \mathbb{E}_{(a,o) \sim \mathcal{D}} \left[\nabla_{\theta} \log h(a|o; \theta) \nabla_{\theta} \log h(a|o; \theta)^T \right], \quad (8)$$

where a is the agent action, o is the agent observation, and both a and o are sampled from the current task $\mathcal{D}$. In practice, $\mathcal{D}$ can be replaced by the experiences E . Denote the i -th eigenvalue of $\mathcal{I}(\theta)$ as λ_i , and the smallest eigenvalue of $\mathcal{I}(\theta)$ as $\lambda_{\min}(\mathcal{I})$.

When the samples in E exhibit the diversity along the basis vector v_i in the task vector space, there exist N samples whose gradient projections onto v_i are non-zero. As expressed in Eq. 9, when the gradient projections $\nabla_{\theta} \log h(a_k|o_k; \theta) \cdot v_i$ of multiple samples along the basis vector v_i are non-zero, the cumulative sum of the squared magnitudes of these samples will increase the eigenvalue λ_i :

$$\lambda_i \propto \sum_{k=1}^N (\nabla_{\theta} \log h(a_k|o_k; \theta) \cdot v_i)^2. \quad (9)$$

The trace of covariance matrix satisfies

$$\text{Tr}(\text{Cov}(\hat{\theta})) \geq \text{Tr}(\mathcal{I}(\theta)^{-1}) = \sum_{i=1}^d \frac{1}{\lambda_i}, \quad (10)$$

where $\text{Tr}(\cdot)$ represents the matrix trace and d denotes the task vector dimension. So, an increase in $\lambda_{\min}(\mathcal{I})$ will lead to a decrease in $\text{Tr}(\text{Cov}(\hat{\theta}))$, which results in a reduction of variance of the estimated task vector θ . Generally, precise estimation of task vector is conducive to the effective decision-making of the LLM-based agent.

In Section III-A1, we have also discussed the limitation of existing methods in reflecting on the historical tasks from one single perspective. In parameter estimation of the task vector, the same kind of experiences E will impede the increase of $\lambda_{\min}(\mathcal{I})$, thereby resulting in the imprecise estimation of task vector $\theta(E)$.

Our designed Dual-Track Experiential Reasoning can reduce the estimation variance of $\theta(E)$ by enhancing the diversity of E , as illustrated in Fig. 3. Specifically, the enhanced $\hat{E}$ consists of the concrete experiences E_{ce} , the abstract experiences E_{ae} , and the action outcome predictions E_{pe} . Dual-Track Experiential Reasoning uses $\hat{E}$ to replace the conventional experiences E , so as to yield a precise task vector $\theta(\hat{E})$. With observation o_i , the probability of the agent to select action a_i is

$$P(a_i|o_i) = f_{\mathcal{M}}(a_i, o_i; \theta(\hat{E})). \quad (11)$$

The remaining parts of this subsection introduce how to obtain concrete experiences, abstract experiences, and action outcome predictions.

1) *Concrete Experiences and Abstract Experiences*: Dual-Track Experiential Reasoning involves both concrete and abstract experiences for reasoning, which is inspired by the Cone of Experience Theory [43]. In this theory, various experiences are divided into different levels according to their abstractness, and human students use these experiences to complete tasks. SRR imitates the process in this theory, dividing the experiences gained from interactions with the environment into concrete experiences and abstract experiences to assist agent in reasoning. In greater detail, the concrete experiences come from the past events, while the abstract experiences are the general concepts derived from these events.

By using LLM, SRR can obtain the concrete experiences from the task trajectories and task outcomes. Here, we define the concrete experiences E_{ce} as

$$E_{\text{ce}} = f_{\mathcal{M}}(\{s_i, a_i\}_{i=1}^n, r, I_{\text{ce}}), \quad (12)$$

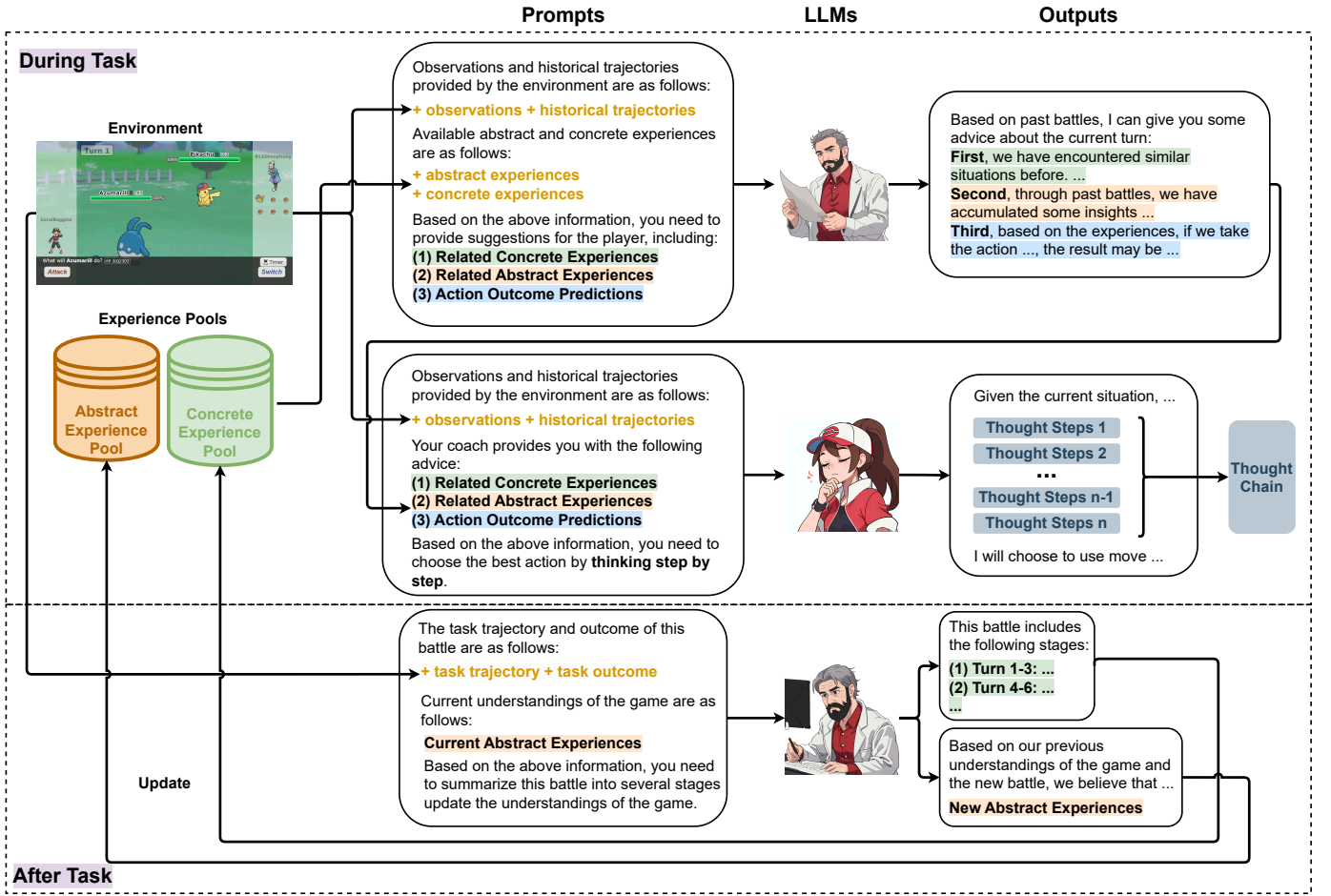


Fig. 3. Illustration of Dual-Track Experiential Reasoning. (a) During the task, the Coach selects the concrete and abstract experiences relevant with the current battle from the experience pools based on the current observations and historical trajectories of agent, and then makes predictions based on these experiences. Subsequently, the Player obtains the observations and trajectories from the environment, and receives the action outcome predictions as well as the concrete and abstract experiences from the Coach. By utilizing these information, the Player takes a step-by-step reasoning process to generate a thought chain. (b) After the task, the Coach refines the concrete experiences specific to the battle, and updates the general understanding of game strategy based on the process and outcome of the battle.

where $\{s_i, a_i\}_{i=1}^n$ denotes the task trajectories, r denotes the task outcomes, and I_{ce} indicates the instruction that prompts the agent to acquire the concrete experiences. SRR generates the new abstract experiences based on both concrete experiences E_{ce} and abstract experiences E_{ae} , which is defined as

$$E_{ae} = f_{\mathcal{M}}(E_{ce}, E_{ae(old)}, I_{ae}), \quad (13)$$

where I_{ae} indicates the instruction that prompts the agent to obtain abstract experiences, and $E_{ae(old)}$ refers to the previous abstract experiences. For intuitive understanding, the examples of concrete and abstract experiences are offered as below.

Concrete Experience

Kingler and Magnezone were taken out due to the opponent's Tentacruel and Turtonator using Sludge Bomb, Rock Slide, and Shell Smash.

Abstract Experience

When the opponent's Pokémon acquires immense power by forgoing its defensive prowess, it becomes vital to utilize strategic moves like Sticky Web to constrain their available choices.

The joint use of concrete and abstract experiences enables an agent to understand the current environment in detail and in depth. The agent can reference the specific past tasks to make decisions, and rely on the general understanding of the task to form the strategy.

2) *Experience and Prediction:* Dual-Track Experiential Reasoning involves both past experiences and action outcome predictions for reasoning, which is inspired by the future-oriented dialogue in the task-oriented context [44].

With an LLM, the agent can predict the action outcomes on the basis of the task trajectories as well as both the concrete and abstract experiences. At timestep i , for each candidate action $a_k \in A$, the agent will predict the outcome of this

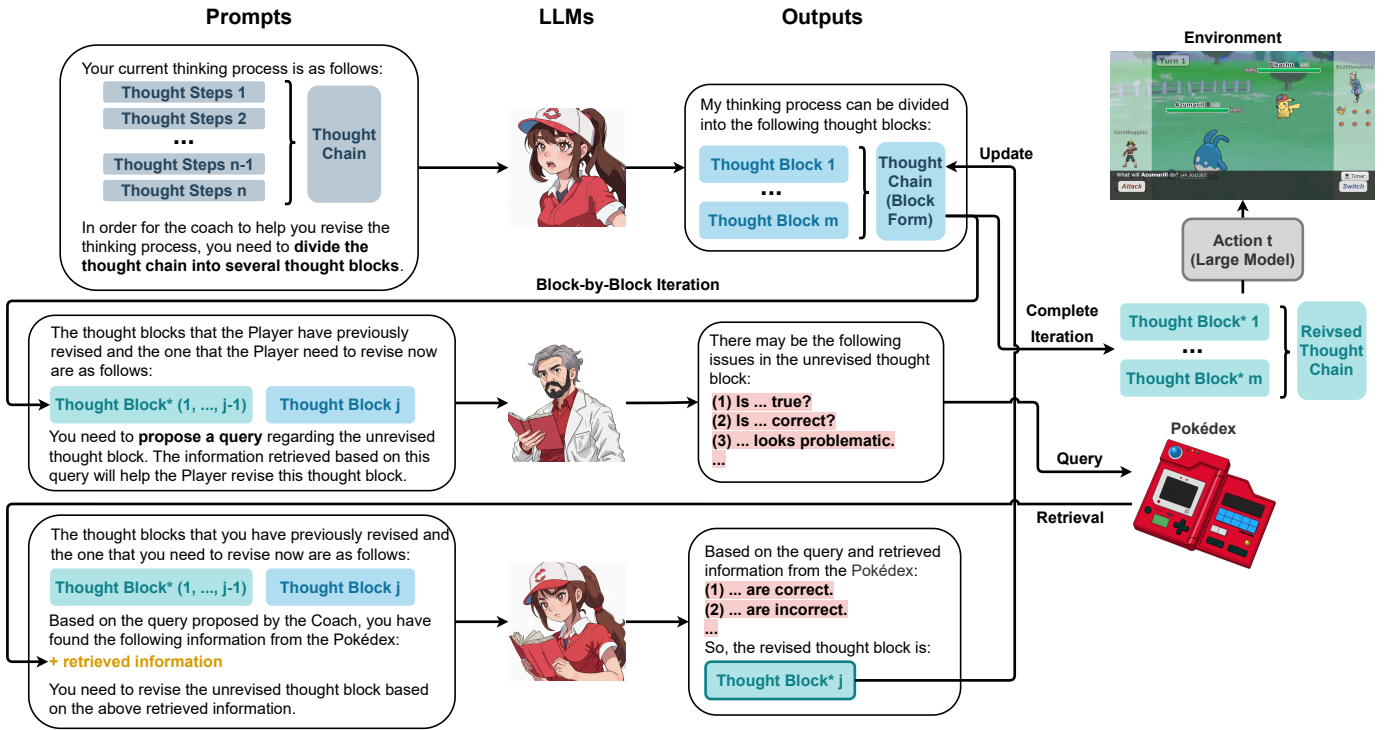


Fig. 4. Illustration of B-RAT. (a) The Player segments the reasoning thought chain into logically-coherent thought blocks. (b) The Coach generates a query for the unrevised thought block, and then retrieves the corresponding data from the database to supply to the Player. (c) The Player revises the thought block according to the retrieved data. After Steps 2 and 3 have iterated through all thought blocks, the final reasoning outcome is formed in the last thought block.

action. The action outcome prediction is formulated as

$$E_{pe}^k = f_{\mathcal{M}} \left(s_i, a_k, \{s_j, a_j\}_{j=i-m}^{i-1}, E_{ce}, E_{ae}, I_{pe} \right), \quad (14)$$

where $\{s_j, a_j\}_{j=i-m}^{i-1}$ stands for the historical trajectory with a length of m , I_{pe} indicates the instruction that prompts the agent to predict the action outcome. After this, the action outcome predictions of m actions are combined into a set $E_{pe} = \{E_{pe}^k\}_{k=1}^m$, which belongs to the enhanced experiences $\hat{E}$. An example of action outcome prediction is given as below.

Prediction

Switching to Klinklang could deal significant damage to Shuckle, but the lingering effect of Sticky Web may slow Klinklang down, making it more vulnerable to enemy.

The prediction comes from both concrete and abstract experiences, hence it is closely related to them both. The concrete experiences can supply the specific historical task data for the agent to predict the action outcomes, but the applicability of such experiences is limited to the past actions of the agent. The abstract experiences can enhance the applicability of these historical task data, so that this agent can better understand the potential outcomes of actions.

E. Block-Retrieval-Augmented Thoughts

In Section III-A2, we interpret the process of generating query as ICL, which is hindered by the conflictive thought

steps. According to Eq. 8-10, such conflict will decrease $\lambda_{\min}(\theta)$, amplify the variance of estimated task vector θ , and finally degrade the effectivity of query q for information retrieval. To handle the textual logical incoherence for query generation, B-RAT harnesses the text comprehension capability of LLM to autonomously adjust the retrieval granularity, as shown in Fig. 4. The textually-coherent thought blocks are beneficial for the precise estimation of task vector θ , which will facilitate the generation of task-relevant query q . Consequently, the retrieved documents are more relevant to the task than before, and thus these documents can be utilized to effectively revise the thought chain to improve the reasoning performance of agent.

Based on the thought chain, B-RAT separates the thought steps into several thought blocks:

$$\{B_i\}_{i=1}^m = f_{\mathcal{M}}(T, I_b), \quad (15)$$

where $B_i = \{t_j\}_{j=1}^n$ denotes the i -th thought block consisting of n thought steps, T represents the thought chain, and I_b indicates the instruction that prompts the agent to separate the thought steps. The thought blocks can serve as the substitutes for the thought steps:

$$q_i = f_{\mathcal{M}}(A, o, B_{1:i-1}^*, B_i, I_q), \quad (16)$$

$$B_{1:i}^* = f_{\mathcal{M}}(A, o, D_i, I_r). \quad (17)$$

where A denotes action set, o denotes the agent observation, and $B_{1:i}^* = \{B_j^*\}_{j=1}^i$ stands for the revised thought blocks. After revising all the thought blocks, B-RAT delivers the received $B_{1:m}^*$ to the prompt of agent for action selection.

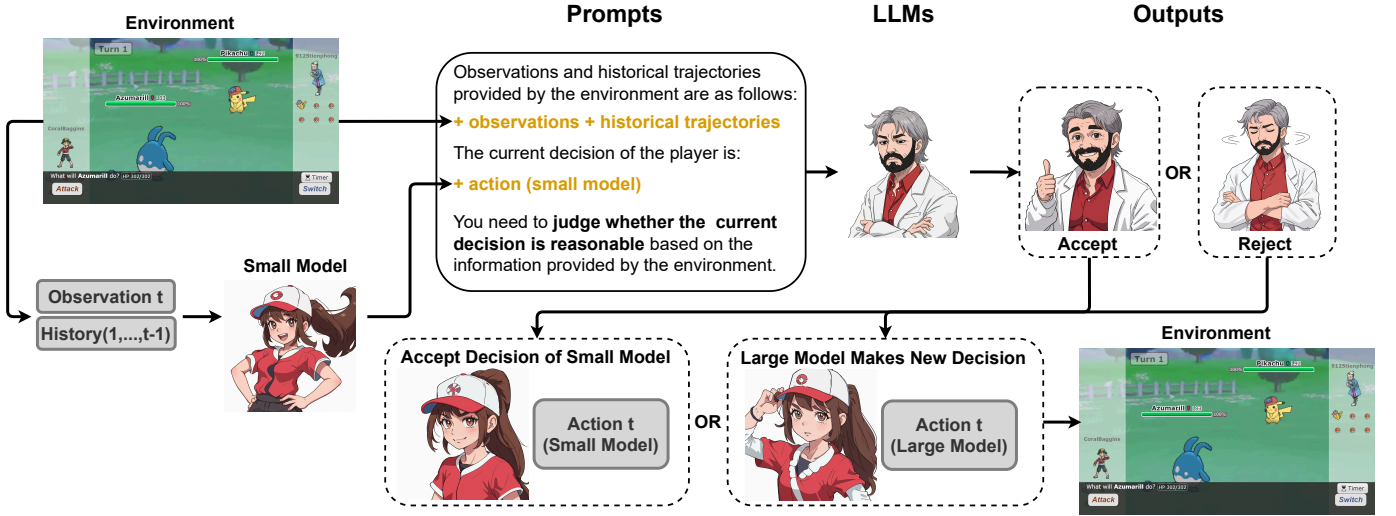


Fig. 5. Illustration of Self-Reliant Thinking System Switch. (a) The Player, guided by System 1 (the small model), takes an action based on the current observation of the agent. (b) The Coach judges whether the action from the Player is reasonable or not. (c) If the action is reasonable, the agent will take the action; otherwise, the Player will switch to System 2 (the large model) for taking a new action.

F. Self-Reliant Thinking System Switch

To lower the computational cost and automate the model switching function, we develop Self-Reliant Thinking System Switch, as shown in Fig. 5. This component dispenses with the heuristic algorithms designed by experts. Instead, it resorts to the large model to judge whether the reasoning outcome of the small model is reasonable or not. Thus, the reasoning process contains the switching steps as follows.

- The small model receives an observation o_i and generates an action a_i .
- The large model receives an observation o_i and an action a_i of the small model, and then judges whether a_i is reasonable or not. The judgment process can be recast as a binary classification problem:

$$P_J(a_i|o_i) = f_{\mathcal{M}}(a_i, o_i, I_J), \quad (18)$$

where $P_J(a_i|o_i)$ represents the probability of action a_i recommended by the large model, and I_J indicates the instruction that prompts the agent to judge the reasonability of action a_i .

- If the large model considers a_i as reasonable, the agent will execute a_i ; otherwise, the large model re-performs the reasoning based on the observation o_i to generate a new action a_i^* .

Given the average reasoning time T_s of the small model, the average reasoning time T_l of the large model, the verification time T_e of the large model, and the acceptance probability p of the action taken by the small model, the autonomous switching between the large and small models can reduce the average decision time of agent if and only if

$$\begin{aligned} (T_s + T_e) \cdot p + (T_s + T_e + T_l) \cdot (1 - p) &< T_l \\ \Rightarrow \frac{T_s + T_e}{T_l} &< p. \end{aligned} \quad (19)$$

When the small model meets the requirement of System 1 in the Dual Process Theory: its average reasoning time T_s

is significantly shorter than T_e of the large model, and its action outputs are reasonable, Eq. 19 will be satisfied. But, when average reasoning time T_s of the small model result in a low acceptance probability p , Eq. 19 will fail to be satisfied. Actually, according to our empirical results in the Pokémon battle environment, T_l (around 25 seconds) for multi-step reasoning exceeds T_e (around 3 seconds) for single-step reasoning, T_s is around 200 milliseconds, and p of the domain-specific small model is around 30%, so Eq. 19 is completely satisfied and thereby the average decision time of agent can be reduced.

In addition, we can adjust the prompt to guide the large model to make strict or lenient judgment, thereby influencing the large model workload in reasoning.

IV. EXPERIMENTAL SETUP

A. Evaluation Environment

We demonstrated the effectiveness and efficiency of SRR-empowered agent in the Pokémon battle environment [14]. This environment interacts with the Pokémon Showdown server [45] and receives game instructions from the simulator [46]. In one-versus-one battles, each player is assigned with six Pokémon: one in the active combat and the others on the bench. In a battle, players take actions in turn. The actions include attacking the opponents Pokémon and replacing the Pokémon with substitute ones. The Pokémon battle environment delivers the critical feedbacks to the players per turn. The battle goal is to defeat all the opponents Pokémon whilst keeping at least one of our Pokémon alive.

The strategic and intricate nature of Pokémon battles renders it a suitable environment for validating the reasoning capability of an agent. The agent needs to consider multiple factors, including the species, types, abilities, statistics, items, and moves of all Pokémon both in active combat and on the bench. Besides, in a randomly generated battle, each Pokémon is selected from a vast pool of over one thousand candidates.

This setup requires the agent to possess not only comprehensive knowledge of Pokémon but also a strong capacity for sophisticated reasoning.

B. Evaluation Metrics

The experiments from Section V-A to Section V-D cover the battles between the LLM-based agent and the default bot empowered by the in-built algorithm in the environment. The evaluation metrics are detailed in the following.

- (a) Win rate and average battle score. As the metrics widely used in the baseline works [14], [47], [48], the win rate is equal to the ratio between the number of victorious battles and the total number of battles, and the battle score is equal to the total number of both fainted opponent Pokémon and un-fainted agent Pokémon at the end of battle, ranging from 0 to 12. These metrics can measure the enhancement of reasoning capability. The enhanced reasoning capability can enable the agent to select the contextually-appropriate actions, which reduces the critical errors and enhances the strategic effectiveness. Consequently, the increase of win rate reflects the ability of agent to achieve the terminal goal, while the elevation of battle score reveals the proficiency of agent in maximizing the cumulative advantage.
- (b) Reasoning speed — the reasoning speed of agent is equal to the average time cost per turn of game. This metric can quantify the efficiency of dynamic inference of the agent. The variability of game states ensures that no historical observation can completely match the current context. Consequently, the agent cannot look up any table but perform real-time reasoning to accommodate the historical experiences with new scenarios.
- (c) Other metrics. The switching frequency, the proportion of panic switching, the number of revisions per action, and the workload of large model are described in Section V-B, V-C and V-D. These metrics cast light on how SRR enhances its reasoning capability: the reduced panic switching minimizes the wrong actions, which can boost the win rate and the average battle score; the small number of revisions per action of the agent can accelerate the reasoning speed; and the workload of large model can affect the balance between the computational latency and the decision quality of the agent.

The experiments in Section V-E involve the battles between the human players and the default bot. Note that we take advantage of the win rate and average battle score to evaluate the performance of human players.

C. Evaluation Details

From Section V-B to Section V-D, except Subsection V-B2, the experimental setup is described in the following: there are five battle sequences, each of which consists of 5 battles; each sequence is repeated for 2 times, and the team configurations within each sequence are different between battles. In Section V-B2, the experimental setup is delineated as follows: there are three battle sequences, each of which comprises 25 battles;

each sequence is repeated for 2 times, and the team configurations within each sequence are the same across battles. As for the experiments in Section V-E, every human player engages in the battles against the default bot in the game environment twice: before and after learning to play Pokémon; all human players take part in one battle sequence, which contains 10 battles, and all battles in each sequence are played once in turn.

D. LLM and Hardware Choices

We tested the agent equipped with different LLMs, including Gemma:2b (Instruct)¹, Gemma:7b (Instruct)², Llama3:8b (Instruct)³, Phi-4-reasoning⁴, GPT-3.5⁵, and GPT-4⁶. The LLM used in each experiment is shown in Table I. For ablation studies, we used Gemma:7b and Llama3:8b on account of their balanced trade-off between the output text quality and the computational overhead. For the ablations on the LLM hallucination, we further use the smaller model Gemma:2b, because Gemma:2b exhibits the pronounced hallucination phenomenon. In the experiments where the LLM agent serves as a human mentor, we selected Llama3:8b, considering that Gemma:2b exhibits the relatively low output text quality for effective mentoring, and Phi-4-reasoning, GPT-3.5 and GPT-4 incur the excessive costs. By contrast, Llama3:8b demonstrated the better reasoning performance and the faster reasoning speed than Gemma:7b.

TABLE I
LLM USED IN EACH EXPERIMENT.

Section	Gemma:2b	Gemma:7b	Llama3:8b	GPT-3.5	GPT-4	Phi-4-reasoning
V-A	✓	✓	✓	✓	✓	✓
V-B		✓	✓			
V-C	✓	✓	✓			
V-D		✓	✓			
V-E			✓			

Notes: ✓ indicates that the LLM is used in the experiment.

For the open-source models, including Gemma:2b, Gemma:7b, Llama3:8b and Phi-4-reasoning, we followed the official recommended practice of Hugging Face. For the closed-source models, including GPT-3.5 and GPT-4, we followed the official recommended practice of LangChain. The key parameters of models have been given in Table II.

Considering the practical deployment of the LLM-based agent on the devices with probably different computational capacities, we conducted experiments using three devices with different computational capacities:

- 1) High-end: GPU of NVIDIA GeForce RTX 4090 (82.58 TFLOPS FP16) and CPU of Intel Core i9-13900KF;

¹<https://huggingface.co/google/gemma-2b-it>

²<https://huggingface.co/google/gemma-7b-it>

³<https://huggingface.co/meta-llama/Meta-Llama-3-8B-Instruct>

⁴<https://huggingface.co/microsoft/Phi-4-reasoning>

⁵<https://platform.openai.com/docs/models/gpt-3.5-turbo>

⁶<https://platform.openai.com/docs/models/gpt-4-turbo>

TABLE II
KEY PARAMETERS OF LLMs.

LLM	Library and Interface	Inference Precision	Task Specification	Temperature	Top_K	Top_P	Max_New_Tokens
Gemma:2b	transformers, AutoTokenizer AutoModelForCausalLM	float16	chat completion	0.7	50	0.9	2048
Gemma:7b	transformers, AutoTokenizer AutoModelForCausalLM	float16	chat completion	0.7	50	0.9	2048
Llama3:8b	transformers, pipeline	float16	text generation	0.6	50	0.9	2048
Phi-4-reasoning	transformers, AutoTokenizer AutoModelForCausalLM	float16	chat completion	0.8	50	0.95	32768
GPT-3.5	LangChain, ChatOpenAI	–	chat completion	–	–	–	–
GPT-4	LangChain, ChatOpenAI	–	chat completion	–	–	–	–

- 2) Mid-tier: GPU of NVIDIA GeForce RTX 4070 Ti SUPER (44.10 TFLOPS FP16) and CPU of Intel Core i5-13490KF;
- 3) Specialized: GPU of NVIDIA GeForce RTX 5090D (104.8 TFLOPS FP16) and CPU of Intel Core i7-265KF.

We resorted to the servers of OpenAI for the closed-source model evaluation (Section V-A). The experiments around Phi-4-reasoning (Section V-A) were carried out based on the specialized configuration. The experiments on model-switching efficiency (Section V-D1 and Section V-D2) employed the mid-tier setup. The human-player experiments (Section V-E) leveraged the different computation devices for different participants. The remaining experiments about the baseline method comparison and the key component ablation were performed based on the high-end configuration.

E. Evaluation on SRR without Switching Function

We evaluated the reasoning capability of SRR-empowered agent irrespective of the reasoning speed by disabling the component of Self-Reliant Thinking System Switch. In this case, the large model is solely responsible for the entire reasoning process. Equipped with each of the six different LLMs, the SRR-empowered agent is compared with the agent adopting Vanilla⁷, Reflexion [16], RAT [25], PreAct [49], PokeLLMon (CoT) [14], PokeLLMon (ToT) [14] and PokeLLMon (SC) [14]. Among these compared methods, PokeLLMon (CoT), PokeLLMon (ToT), and PokeLLMon (SC) belong to the baseline methods in the Pokémon battle environment. These experiments assess the method ability to take advantage of the accumulated experiences to mitigate the hallucination errors. The experimental results have been provided in Section V-A.

F. Ablation on Dual-Track Experiential Reasoning

In this ablation study, we address the first research question: can guiding the LLM-based agent to reflect from multiple perspectives benefit its behavioral strategy? To this end, we delve into what roles do the different experiences play in enhancing the agent reasoning capability when using Dual-Track Experiential Reasoning.

⁷In Vanilla, the agent makes decisions on the basis of immediate observations without any revision.

We examined the impact of the key ingredients, including concrete experiences, abstract experiences, and action outcome prediction, in Dual-Track Experiential Reasoning on SRR under variable team configurations. Specifically, Reflexion merely utilizes either of the concrete experiences or abstract experiences, both of which have been paid attention to by SRR. PreAct only predicts the action outcome, which is also done by SRR. Therefore, these experiments can help verify the advantage of SRR over Reflexion and PreAct. The experimental results have been offered in Section V-B1.

Besides, we investigated whether the SRR-empowered agent can learn useful experiences and achieve ultimate victory in the complex scenarios by means of the repeated interactions with the environment under fixed team configurations. Meanwhile, we evaluated the impact of key ingredients, including concrete experiences, abstract experiences, and action outcome prediction, in Dual-Track Experiential Reasoning on the convergence speed of SRR. The experimental results have been supplied in Section V-B2.

G. Ablation on Block-Retrieval-Augmented Thoughts

In this ablation study, we address the second research question: can the LLM-based agent leverage its text analysis ability to autonomously determine the retrieval granularity for maintaining the textual logical coherence? For this purpose, we probe into how the revision of reasoning outcome and process affects the effectivity and efficiency of reasoning when using B-RAT.

We compare B-RAT with two typical methods, RAG [34] and RAT [25]. Specifically, RAG represents a direct revision on the outcome of reasoning process. RAT revises every thought step in the thought chain. At the same time, we measured the reasoning efficiency of these methods by means of the number of revisions per action. The experimental results have been given in Section V-D.

H. Ablation on Self-Reliant Thinking System Switch

In this ablation study, we address the third research question: can the LLM-based agent achieve autonomous model switching based on the intrinsic reasoning capability, without using any predefined rule? For this aim, we look into whether the preference of autonomous model switching can be controlled

by adjusting the prompt when using Self-Reliant Thinking System Switch.

We evaluated the impact of switching between the small and large models and meanwhile measured the reasoning speed based on the time cost per turn of game. In this experiment, the small model always selects the action with the highest power from the available actions of Pokémon. Meanwhile, we compared the component of Self-Reliant Thinking System Switch with a heuristic switching method, which adopts the reward function from PokeLLMon [14]. For the heuristic switching method, if an agent using the small model receives negative rewards for three consecutive actions, this agent will switch to using the large model for the subsequent three actions. The experimental results have been detailed in Section V-D1.

Besides, in order to control the preference for autonomous switching, we inform the used LLM of the roles that it is playing in SRR, 'strict' or 'lenient'. The experimental results have been displayed in Section V-D2.

I. LLM-Based Agent as Mentor for Novice Players

At last, we address the fourth exploratory question: can the LLM-based agent with the enhanced reasoning capacity serve as a mentor for novice players? For this goal, we explore the feasibility of using the SRR-empowered agent to guide novice human players in playing Pokémon battle games, and compare the guiding effect of the SRR-empowered agent with the means of tutorial study.

The experiments involved 21 participants, who were informed of the content of experiments and the extent of their participation. The written informed consent was collected from all these participants. The consent includes the confidentiality of their data and the voluntariness of their participation. In addition, all the experiments were conducted in accordance with relevant guidelines and regulations.

We chose Llama3:8b as the LLM for the SRR-empowered agent. Here, we disable the function of switching between small and large models, because the small model cannot offer any explanatory information in the process of reasoning.

Based on a baseline test, the participants were split into three groups: an LLM group, a non-LLM group, and a control group, with two video game novices and five experienced players who had been unfamiliar with Pokémon in each group. There are three stages for experiments as below and the experimental results have been exhibited in Section V-E.

- (a) Baseline Test: Participants joined in ten battles against the built-in bot to assess their gaming skill levels in terms of win rate and average battle score before the learning stage. This test ensured an even distribution of gaming skills among the three groups.
- (b) Learning Stage: The LLM group learned how to win the game by analyzing the reasoning processes of the SRR-empowered agents. The non-LLM group learned how to win the game using the document tutorials and video tutorials. Meanwhile, the control group merely relied on the direct gameplay experiences to learn the strategy in the Pokémon game. Every participant of all the groups took

part in 50 battles. Additionally, for the non-LLM group, viewing a full battle commentary was regarded as joining in one battle as well.

- (c) Post Test: This test involved ten battles for every participant against the built-in bot, with win rates and average battle scores recorded for comparative analysis.

Based on the behavioral learning theory [50], cognitive learning theory [51] and motivation theory [52] in the context of video games [53], the human learning process of participants should encompass the ability advancements in the following three aspects.

- (a) Knowledge acquisition. Players understand the game rules, such as the most effective actions for certain types of Pokémon.
- (b) Skill development. Players become familiar with the battle system of the environment, such as arranging the sequence of Pokémon actions reasonably.
- (c) Strategy refinement. Players enhance the ability of long-term planning, such as sacrificing short-term profits to preserve the critical resources.

V. EXPERIMENTAL RESULTS

A. Evaluation on SRR without Switching Function

As shown in Fig. 6, Reflexion [16], RAT [25], PreAct [49], PokeLLMon (CoT) [14], PokeLLMon (ToT) [14] and PokeLLMon (SC) [14] acquire the suboptimal win rates and average battle scores, primarily due to their limited utilization of experiences or an inadequate revision mechanism. In contrast, in SRR, Dual-Track Experiential Reasoning fully utilizes experiences, and B-RAT maintains the textual logical coherence in reasoning. The results in Fig. 6 show that the SRR-empowered agent can consistently achieve higher win rates and average battle scores than the compared methods, which validates the strong reasoning capability of the SRR-empowered agent. Besides, as reported in Table III, even when the component of Self-Reliant Thinking System Switch is disabled, the SRR-empowered agent can still have a moderate reasoning speed. We believe that, with the fast development of hardware, such as GPU and CPU, the reasoning of the SRR-empowered agent will be much accelerated in the not-so-far future.

B. Ablation on Dual-Track Experiential Reasoning

1) *Variable Team Configurations*: The results in Table IV verify that the agent achieves peak win rate and highest average battle score when incorporating all three experiences. The ablation of any experience results in degradation across both win rate and average battle score.

As shown in Table V, further analysis of the experiment process reveals that, when executing the blind prediction, the agent exhibits heightened frequency of 'panic switching'. Panic switching is a phenomenon introduced by PokeLLMon [14], which means that an agent continuously switches between different Pokémon to evade from the battle. In Table V, the metric 'Switching Frequency' refers to the percentage of switching actions among all agent actions, while 'Proportion of Panic Switching' indicates how many of these switching actions are panic switching.

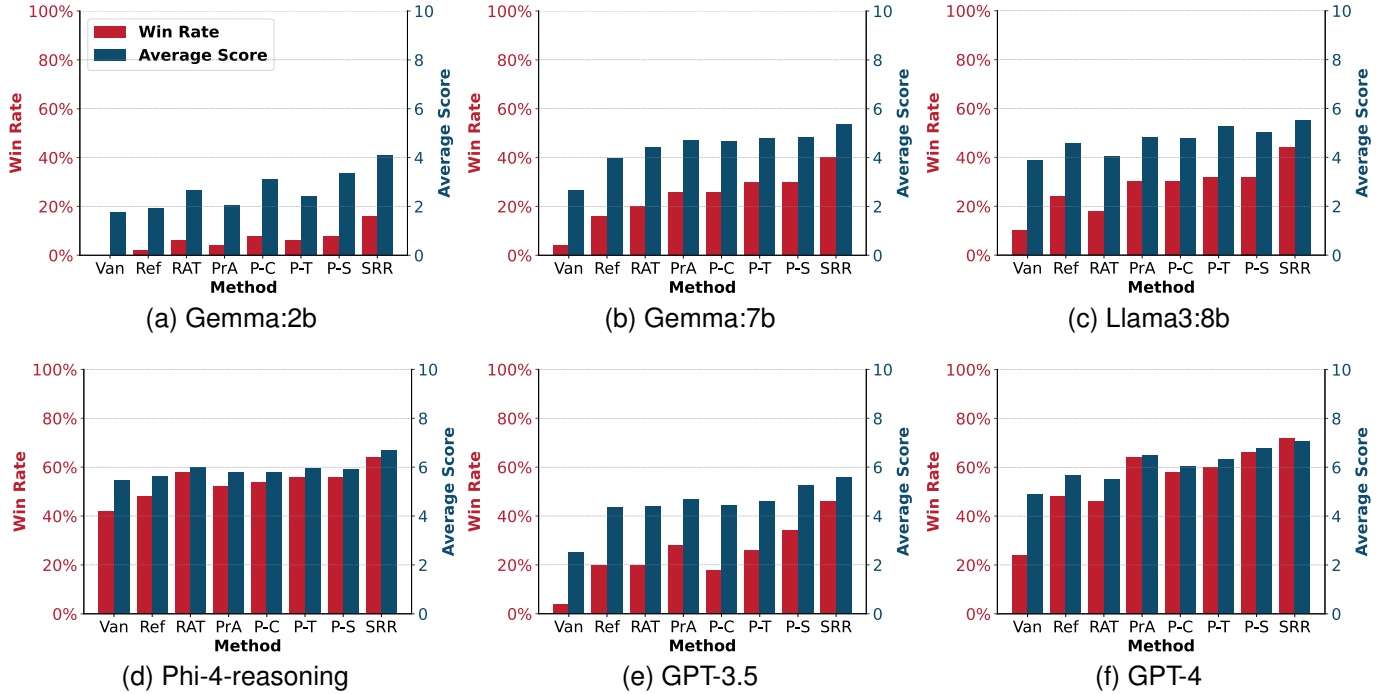


Fig. 6. Method comparison. ‘Van’ denotes Vanilla; ‘Ref’ denotes Reflexion; ‘PrA’ denotes PreAct; ‘P-C’, ‘P-T’, and ‘P-S’ indicate PokeLLMon (Chain-of-Thought), PokeLLMon (Tree-of-Thought), and PokeLLMon (Self-Consistency), respectively.

TABLE III
EVALUATION ON THE REASONING SPEED OF THE LLMs FOR SRR.

LLM for SRR	Method	Time Cost	LLM for SRR	Method	Time Cost
Gemma: 2b	Van	6.97	Gemma: 7b	Van	17.72
	Ref	7.96		Ref	19.83
	RAT	10.86		RAT	22.35
	PrA	10.23		PrA	21.53
	P-C	10.77		P-C	20.49
	P-T	14.42		P-T	29.71
	P-S	11.46		P-S	23.68
	SRR	12.20		SRR	23.52
Llama3: 8b	Van	15.08	Phi-4-reasoning	Van	20.10
	Ref	16.74		Ref	25.14
	RAT	21.15		RAT	32.75
	PrA	17.96		PrA	31.38
	P-C	18.90		P-C	28.61
	P-T	25.47		P-T	41.22
	P-S	19.93		P-S	33.23
	SRR	20.12		SRR	35.88

Notes: ‘Time Cost’ denotes the time cost (seconds) per turn.

TABLE IV
EVALUATION ON THE KEY INGREDIENTS OF DUAL-TRACK EXPERIENTIAL REASONING ON SRR UNDER VARIABLE TEAM CONFIGURATIONS.

LLM for SRR	Reasoning Model	Win Rate	Average Score	Time Cost
Gemma: 7b	$E_a + E_c + E_p$	42.00%	5.36	22.87
	$E_c + E_p$	32.00%	4.80	21.04
	$E_a + E_p$	38.00%	5.20	20.96
	$E_a + E_c$	38.00%	5.22	19.36
	E_p	30.00%	4.62	21.93
	None	30.00%	4.80	17.84
Llama: 8b	$E_a + E_c + E_p$	46.00%	5.64	21.43
	$E_c + E_p$	36.00%	5.04	20.47
	$E_a + E_p$	40.00%	5.36	20.86
	$E_a + E_c$	42.00%	5.48	17.23
	E_p	30.00%	4.60	19.05
	None	32.00%	4.88	15.67

Notes: ‘None’ indicates that SRR does not rely on any experience or prediction for reasoning; the highest performances of all the evaluated methods are marked in bold.

2) *Identical Team Configurations*: Fig. 7 and Table. VI and reveals that, by accumulating the battling experiences, the SRR-empowered agent can defeat the opponents that were once hard to defeat. When the reasoning process lacks abstract experiences or action outcome predictions, the convergence speed of average battle score decreases. Notably, the absence of concrete experiences largely impedes the growing of average battle score of the SRR-empowered agent.

C. Ablation on Block-Retrieval-Augmented Thoughts

The results in Table VII confirm that, within SRR, B-RAT not only can enhance the reasoning capability of an agent via revising the intermediate steps in reasoning, but also can help reduce the number of revisions necessary for the entire thought chain. But, we can also observe that the agent equipped with Gemma:2b fails to yield significant performance improvement when transitioning from RAT to B-RAT.

TABLE V
EVALUATION ON THE PANIC SWITCHING IN SRR WITH AND WITHOUT BLIND PREDICTION.

LLM for SRR	Reasoning Model	Average Score	Time Cost	Switching Frequency	Proportion
Gemma: 7b	E_p	4.62	21.93	30.50%	14.20%
	None	4.80	17.84	21.60%	6.22%
Llama3: 8b	E_p	4.60	19.05	32.60%	17.42%
	None	4.88	15.67	24.80%	5.14%

Note: 'Proportion' denotes proportion of panic switching.

TABLE VI
EVALUATION ON THE REASONING SPEED OF THE LLMs FOR SRR UNDER IDENTICAL TEAM CONFIGURATION.

LLM for SRR	Reasoning Model	Time Cost	LLM for SRR	Reasoning Model	Time Cost
Gemma: 7b	$E_a+E_c+E_p$	23.12	Llama3: 8b	$E_a+E_c+E_p$	21.33
	E_c+E_p	20.92		E_c+E_p	20.72
	E_a+E_p	20.85		E_a+E_p	20.56
	E_a+E_c	19.05		E_a+E_c	18.03

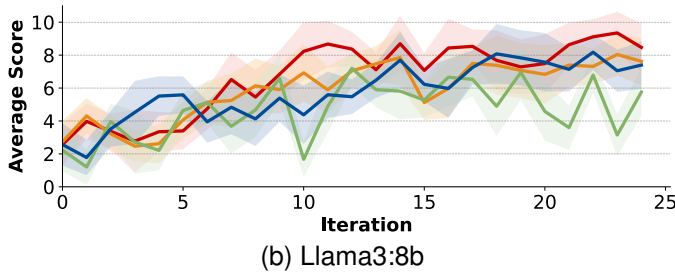
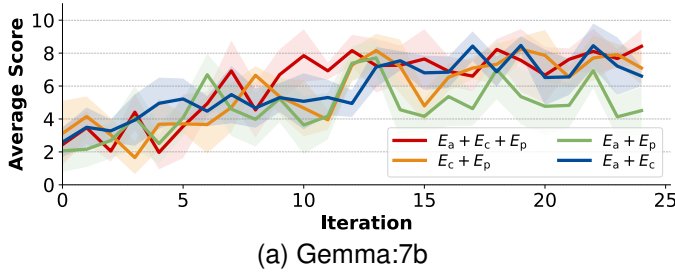


Fig. 7. Evaluation on the key ingredients of Dual-Track Experiential Reasoning on SRR under fixed team configurations.

D. Ablation on Self-Reliant Thinking System Switch

1) *Switching Methods Comparison*: The results in Table VIII validate that Self-Reliant Thinking System Switch can reduce the average reasoning time whilst maintaining the reasoning quality of SRR. To complete a task, although the heuristic switching method has higher efficiency than the self-reliant switching one, the heuristic switching method leads to a significant decrease in the win rate and average battle score. Besides, the results in Table VIII also verify that the device with lower computational capacity will benefit more from the switching method.

2) *Control Switching Preference*: From the results shown in Table IX, we can see that the prompt adjustment has an

TABLE VII
EVALUATION ON THE REVISION METHODS OF SRR WITH DIFFERENT LLMs.

LLM for SSR	Revision Method	Win Rate	Average Score	Revisions	Time Cost
Gemma: 2b	None	4.00%	3.02	0.00	6.86
	RAG	10.00%	3.64	1.00	7.53
	RAT	14.00%	3.78	2.63	10.62
	B-RAT	14.00%	3.84	(2.25, 3.25)	10.81
Gemma: 7b	None	24.00%	4.80	0.00	17.21
	RAG	28.00%	5.00	1.00	18.74
	RAT	34.00%	5.16	6.41	21.95
	B-RAT	38.00%	5.44	(3.48, 4.48)	19.86
Llama3 8b	None	28.00%	5.04	0.00	15.08
	RAG	30.00%	5.12	1.00	16.72
	RAT	38.00%	5.40	5.98	21.15
	B-RAT	44.00%	5.62	(3.13, 4.13)	19.33

Notes: 'None' represents that no revision is made; $(\cdot, \cdot)$ represents the open interval, where the left value is the number of revisions for each action of agent and the right one is the number of revisions for each action of agent plus an additional segmentation step, which enhances the computational cost smaller than one revision. 'Revisions' denotes the number of revisions per action of the agent.

obvious influence on the workload of large model, which is equal to the ratio between the actions generated by this large model and the total number of actions. The LLM playing the 'strict' role tends to increase the workload of large model, leading to higher win rate and average battle score, and longer decision latency, while the LLM playing the 'lenient' is the opposite.

E. LLM-Based Agent as Mentor for Novice Players

The results of baseline test and post test have been visualized in Fig. 8. These results reveal that the participants who utilized external materials, i.e. those outside the control group, can generally achieve superior learning outcomes compared to the control group. The results also manifest that the participants who learned to play the game using the SRR-empowered agent can achieve the comparable learning performances to those who learned to play the game via document and video tutorials. Therefore, we preliminarily believe that, it is efficacious to use the SRR-empowered agent to assist the novice players in learning how to play the Pokémon battle game.

VI. DISCUSSION

A. Experiences in Dual-Track Experiential Reasoning

From Table IV, we find that the multi-perspective reflection can help enhance the reasoning capability of SRR-empowered agent, and any absence of abstract experiences, concrete experiences, and action outcome predictions will weaken the reasoning capability of agent. The weakness caused by the lack of abstract experiences is particularly evident in the variable team configurations. The reason for this phenomenon is explained here. Since the probability of an agent encountering the same or similar environment is quite low in different

TABLE VIII
EVALUATION ON THE SWITCHING METHODS OF SRR USING DIFFERENT LLMs AND HARDWARES.

LLM for SRR	Hardware	Switching Methods	Win Rate	Average Score	Time Cost	T_s	T_e	T_l
Gemma:7b	RTX 4070 Ti SUPER	None	42.00%	5.42	30.22	–	–	30.22
		Heuristic Switch	32.00%	5.04	17.65	0.49	0.71	31.18
		Self-Reliant Switch	40.00%	5.30	21.42	0.52	2.55	30.54
	RTX 4090	None	42.00%	5.48	23.41	–	–	23.41
		Heuristic Switch	34.00%	5.10	15.83	0.44	0.68	23.21
		Self-Reliant Switch	40.00%	5.26	18.76	0.41	1.98	23.96
Llama3:8b	RTX 4070 Ti SUPER	None	44.00%	5.54	28.01	–	–	28.01
		Heuristic Switch	34.00%	5.16	16.21	0.53	0.62	27.86
		Self-Reliant Switch	40.00%	5.34	18.26	0.36	2.43	28.12
	RTX 4090	None	42.00%	5.50	19.20	–	–	19.20
		Heuristic Switch	34.00%	5.20	11.74	0.39	0.64	19.22
		Self-Reliant Switch	42.00%	5.36	15.38	0.41	1.85	20.04

Notes: ‘None’ indicates that the large model performs the entire reasoning. T_s denotes the average reasoning time of the small model. T_e denotes the average verification time of the large model. T_l denotes the average reasoning time of the large model.

TABLE IX
EVALUATION ON THE PROMPT IN SELF-RELIANT SWITCHING OF SRR WITH DIFFERENT LLMs.

LLM for SRR	Switching Methods	Win Rate	Average Score	Time Cost	Workload of Large Model	T_s	T_e	T_l
Gemma:7b	None	42.00%	5.42	30.22	100.00%	–	–	30.22
	Original	40.00%	5.30	21.42	67.72%	0.52	2.55	30.54
	Strict	40.00%	5.40	25.65	81.53%	0.51	2.43	29.97
	Lenient	38.00%	5.26	19.34	58.11%	0.39	2.65	30.82
Llama3:8b	None	44.00%	5.54	28.01	100.00%	–	–	28.01
	Original	40.00%	5.34	18.26	65.19%	0.36	2.43	28.12
	Strict	44.00%	5.48	22.13	77.17%	0.45	2.29	27.88
	Lenient	40.00%	5.30	16.99	60.29%	0.53	2.36	27.79

Notes: ‘Original’ indicates that there is no tendency towards strictness or leniency for the prompt to switch between the small and large models.

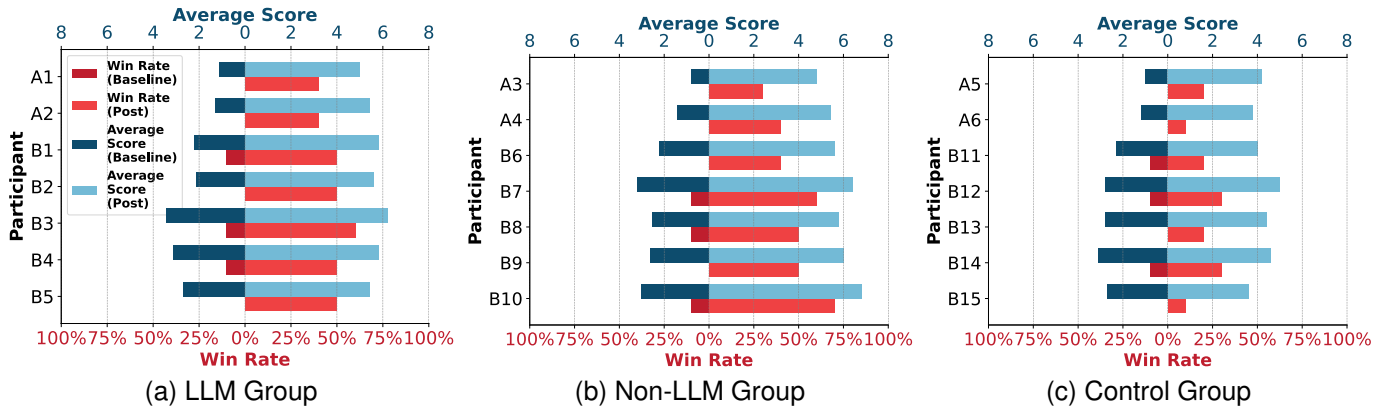


Fig. 8. Comparison of human learning performances between participants in the baseline and post tests. Participants with numbers starting with A are novices in video games, while participants with numbers starting with B are experienced in video games but unfamiliar with Pokémon. Participants in the LLM group used LLM-based agents to learn how to win the game. The LLM-based agents are empowered by the SRR framework without the switching function. Participants in the non-LLM group learned how to win the game using document and video tutorials. Participants in the control group merely relied on the personal gameplay experiences to learn how to win the game.

battles, solely relying on the concrete experiences inevitably limits the ability of agent to predict the action outcomes new scenarios. While, the abstract experiences typically contain the essential knowledge of general gaming strategies among scenarios. Such experiences can enable an agent to sufficiently

make use of the common strategy knowledge of the battles, rather than merely relying on the knowledge from some concrete, independent events, to win the game.

Table V reveal that the blind prediction induces a surge in the frequency of panic switching for an agent. This causally

leads to an obvious performance decline in both win rate and average battle score. From Table V, we can see that, without the blind prediction during the reasoning of SRR, the panic switching occupies around one-third of the total number of switches. When SRR adopts the blind prediction, the proportion of panic switching rises to almost half of the total number of switches. These results indicate that the blind prediction from SRR, lacking the battling experiences, has incurred an overestimation of opponent strength. As a consequence, the agent tries to avoid the battles via Pokémon switching. Nevertheless, the continuous switching also deprives the agent of the opportunities to take actions, consequently resulting in the failure of entire battle.

Fig. 7 exhibits that, based on the same team configuration, the lack of concrete experiences incurs a performance decline of the SRR-empowered agent. By analyzing the battling process, we see that if the concrete experiences are ignored, the agent will be highly prone to repeating the previous mistakes. This finding substantiates the importance for an agent to study from the specific historical events, especially when this agent confronts the same or similar environment.

B. Influence of B-RAT on Different LLMs

As shown in Fig. 6, when using SRR, the reasoning capability of the agent equipped with Gemma:2b is weaker than the one equipped with either Gemma:7b or Llama3:8b. Such results can be traced to the fundamental reasoning ability of LLM [54]. According to Table VII, the agent equipped with Gemma:2b, which has relatively weak reasoning capability, benefits greatly from the revisions of reasoning process. Typically, applying RAG to the reasoning outcome of the SRR-empowered agent, which is equipped with Gemma:2b, can double its win rate. This is mainly because, for the agent equipped with Gemma:2b, the hallucination problem largely hampers the reasoning capability of SRR. But, for the agent equipped with Gemma:7b, the increments of both win rate and average battle score brought by B-RAT for SRR are quite similar to RAT, though B-RAT needs some more revisions for each action. This is chiefly because, compared with the agent equipped with either Gemma:7b or Llama3:8b, both of which have relatively strong reasoning capability, the agent equipped with Gemma:2b can generate the shorter thought chain. The shorter thought chain has the advantage in keeping the textual logical coherence and reducing the number of revisions for the SRR-empowered agent. Therefore, in SRR, the component of B-RAT with Gemma:2b is not definitely superior to RAT, and using RAT alone suffices to making SRR yield reliable outcomes.

C. Effectiveness and Controllability of Self-Reliant Thinking System Switch

The results in Table VIII validate the enhanced effectiveness for the SRR-powered agent when the switching technique is deployed on a computation-constrained device. This kind of device needs enough time for completing all the thought steps of the large model, so lowering the workload of LLM can largely boost the reasoning speed of agent. Such results imply

the great value of SRR for the practical deployment of LLM-based agent in many computation-constraint scenarios.

By observing the game videos in the experiments, we can find the cases that Eq. 19 does not hold. As recorded in Table. X, in a game where all the Pokémon are not good at attacking, the small model always selects the ineffective actions. Hence, the acceptance probability p , as formulated in Eq. 19, is over low, which causes the inequality of Eq. 19 to fail. In this case, the reasoning speed of the agent degenerates to $T_s + T_e + T_l$. This speed is slower than the reasoning speed T_l in the case without using any switching technique. However, such a phenomenon can be mitigated if the agent uses an effective small model.

TABLE X
EXAMPLE OF THE SWITCHING METHODS OF SRR.

LLM for SRR	Hardware	Switching Techniques	Time Cost	T_s	T_e	T_l
Gemma: 7b	RTX 4070 Ti SUPER	None	29.46	–	–	29.46
		Self-Reliant Switch	33.00	0.56	2.97	29.48
	RTX 4090	None	22.92	–	–	22.92
		Self-Reliant Switch	26.26	0.39	1.72	24.16
Llama3: 8b	RTX 4070 Ti SUPER	None	27.56	–	–	27.56
		Self-Reliant Switch	30.04	0.40	2.73	26.91
	RTX 4090	None	17.41	–	–	17.41
		Self-Reliant Switch	21.78	0.33	1.69	19.77

The results in Table IX verify that, compared to the LLM that has not been instructed to play any role, the LLM playing the ‘strict’ role tends to consider the reasoning outcomes of the small model as unreasonable. As a result, the large model undertakes more workload, which increases the reasoning quality but decreases the reasoning speed for the SRR-empowered agent. By contrast, the LLM playing the ‘lenient’ role is inclined to accept the reasoning outcomes of the small model, which can expedite the reasoning process but at the expense of reasoning quality of the SRR-empowered agent. This finding enables us to tailor the prompt of LLM to the requirement of balancing between the reasoning effectivity and efficiency in broad practical scenarios.

D. Advantages of LLM-based Agent as Player Mentor

Fig. 8 shows that the participants in the control group (without external data supplementation) exhibit the suboptimal learning performances. We tentatively explain this phenomenon by the Experiential Learning Theory [55]: the human learners who have access to the external resources own an advantage in understanding the concepts in games, and such an advantage can assist them in getting the essential knowledge of game quickly.

By comparing the performances of the participants using the SRR-empowered agent with the people using the document



Fig. 9. Screenshot of HearthArena plugin. The red-boxed scores within the frame are the card scores derived from human knowledge and expertise. Human players can select the cards based on these scores in the game.

tutorials and video tutorials, we can confirm that, though the strategy supplied by the agent is not always so effective as those presented in the tutorials, the SRR-empowered agent has a unique superiority in offering the tailored feedbacks specific to every individual gameplay environment. This superiority facilitates the novice players to receive the assistance beyond the fixed circumstances in the tutorials, which is conducive to the training of video game skills and the updating of gaming experiences for the human players.

In addition to the document tutorials and video tutorials, there are also scoring plugins available for the novice players to learn how to play the video games. As exemplified in Fig. 9, the HearthArena plugin for the Arena mode in Hearthstone can recommend the appropriate cards to the players on the basis of card scores [56]. Nevertheless, this plugin only can guide human players in the form of numbers. By contrast, the SRR-empowered agent has two distinctive merits. Firstly, it can communicate with the human players in natural language, which is consistent with the regular learning habit of human beings; secondly, such an agent can explain the comprehensible rationale behind its recommendation. According to the Metacognitive Theory [57], elucidating the rationale behind a guidance is quite favorable for the human players to master the metacognitive knowledge of gaming strategy.

VII. CONCLUSION

In this paper, we have proposed the new LLM-based framework, SRR, to enhance the reasoning capability of the agent in video games. In SRR, we have designed the component of Dual-Track Experiential Reasoning, which not only integrates the concrete experiences with abstract ones but also combines the integrated experiences and action outcome predictions together, for enhancing the reasoning capability of agent in complex tasks; we have devised the component of B-RAT which segments the thought chain into multiple thought blocks and generates the task-relevant query for external sources, thus alleviating the hallucination problem in agent reasoning; we have also developed the component of Self-Reliant Thinking System Switch to increase the reasoning speed and and automate the model switching function. Experimental results

have demonstrated that the SRR framework greatly outperforms the related advanced methods equipped with different LLMs. Moreover, the results have validated the importance and indispensability of all the three components in SRR for enhancing the reasoning capability of agent. Furthermore, the experiments in guiding human players have confirmed the efficacy of the SRR-empowered agent to assist in learning how to play the Pokémon battle game. Although SRR has yielded promising outcomes, it has limitations. Unlike humans, who differently leverage successful and unsuccessful experiences in new tasks, SRR currently does not distinguish between these types of experiences. In future, we plan to improve SRR by distinguishing the successful and failed experiences before utilization, and extend the proposed framework to broader application scenarios.

REFERENCES

- [1] G. N. Yannakakis and J. Togelius, *Artificial Intelligence and Games*. Springer, 2018.
- [2] M. Cetin and Y. Sarica, "Artificial intelligence based game levelling," *Balkan Journal of Electrical and Computer Engineering*, vol. 8, no. 2, pp. 147–153, 2020.
- [3] M. F. Syahputra, A. Arippa, R. F. Rahmat, and U. Andayani, "Historical theme game using finite state machine for actor behaviour," in *Journal of Physics: Conference Series*. Sumatera Utara, Indonesia: IOP Publishing, Sep. 2018, pp. 121–127.
- [4] M. Nicolau, D. Perez-Liebana, M. O'Neill, and A. Brabazon, "Evolutionary behavior tree approaches for navigating platform games," *IEEE Transactions on Games*, vol. 9, no. 3, pp. 227–238, 2016.
- [5] W. Jiang, K. Hwang, and J. Lin, "An experience replay method based on tree structure for reinforcement learning," *IEEE Transactions on Emerging Topics in Computing*, vol. 9, no. 2, pp. 972–982, 2021.
- [6] B. Yang, C. Ma, and X. Xia, "An interrelated imitation learning method for heterogeneous drone swarm coordination," *IEEE Transactions on Emerging Topics in Computing*, vol. 10, no. 4, pp. 1704–1716, 2022.
- [7] W. Li, Z. Qiu, S. Shao, and A. Song, "MDDP: Making decisions from different perspectives in multi-agent reinforcement learning," *IEEE Transactions on Games*, vol. 16, no. 3, pp. 621–634, 2024.
- [8] W. Li, W. Liu, S. Shao, S. Huang, and A. Song, "Attention-based intrinsic reward mixing network for credit assignment in multiagent reinforcement learning," *IEEE Transactions on Games*, vol. 16, no. 2, pp. 270–281, 2024.
- [9] W. Li, S. Huang, Z. Qiu, and A. Song, "GAILPG: Multi-agent policy gradient with generative adversarial imitation learning," *IEEE Transactions on Games*, vol. 17, no. 1, pp. 62–75, 2025.
- [10] J. Light, M. Cai, S. Shen, and Z. Hu, "AvalonBench: Evaluating LLMs playing the game of Avalon," in *NeurIPS 2023 Foundation Models for Decision Making Workshop*, New Orleans, United States, Dec. 2023, pp. 23 241–23 252.
- [11] Y. Xu, S. Wang, P. Li, F. Luo, X. Wang, W. Liu, and Y. Liu, "Exploring large language models for communication games: An empirical study on Werewolf," *arXiv:2309.04658*, 2023.
- [12] Z. Wang, S. Cai, G. Chen, A. Liu, X. Ma, and Y. Liang, "Describe, explain, plan and select: Interactive planning with large language models enables open-world multi-task agents," in *Advances in Neural Information Processing Systems*, Los Angeles, USA, Nov. 2023, pp. 34 153–34 189.
- [13] W. Ma, Q. Mi, X. Yan, Y. Wu, R. Lin, H. Zhang, and J. Wang, "Large language models play StarCraft II: Benchmarks and a chain of summarization approach," in *Advances in Neural Information Processing Systems*, Vancouver, Canada, Dec. 2024, pp. 133 386–133 442.
- [14] S. Hu, T. Huang, and L. Liu, "PokeLLMon: A human-parity agent for Pokémon battles with large language models," *arXiv:2402.01118*, 2024.
- [15] R. Banfi, "Gaming I, II, and III: Arcades, video game systems, and modern game streaming services," *Games and Culture*, vol. 19, no. 8, pp. 1000–1037, 2024.
- [16] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems*, New Orleans, United States, Dec. 2023, pp. 8634–8652.

- [17] Z. Gou, Z. Shao, Y. Gong, Y. Shen, Y. Yang, N. Duan, and W. Chen, "CRITIC: Large language models can self-correct with tool-interactive critiquing," *arXiv:2305.11738*, 2023.
- [18] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhunoy, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark, "Self-Refine: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems*, New Orleans, United States, Dec. 2023, pp. 46 534–46 594.
- [19] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-Consistency improves chain of thought reasoning in language models," in *Proceedings of the International Conference on Learning Representations*, Kigali, Rwanda, May. 2023, pp. 1–24.
- [20] E. Shliakhovchuk, "Video games as awareness raisers, attitude changers, and agents of social change," *International Journal of Computer Games Technology*, vol. 2024, no. 1, pp. 1–10, 2024.
- [21] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, and T. Liu, "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," *ACM Transactions Information Systems*, vol. 43, no. 2, pp. 1–55, 2025.
- [22] X. Ma, Y. Gong, P. He, H. Zhao, and N. Duan, "Query rewriting in retrieval-augmented large language models," in *Proceedings of the Conference on Empirical Methods in Natural Language Processing*, Singapore, Dec. 2023, pp. 5303–5315.
- [23] H. S. Zheng, S. Mishra, X. Chen, H.-T. Cheng, E. H. Chi, Q. V. Le, and D. Zhou, "Take a Step Back: Evoking reasoning via abstraction in large language models," in *Proceedings of the International Conference on Learning Representations*, Vienna, Austria, May 2024, pp. 1–38.
- [24] L. Gao, X. Ma, J. Lin, and J. Callan, "Precise zero-shot dense retrieval without relevance labels," in *Proceedings of the Annual Meeting of the Association for Computational Linguistics*, Toronto, Canada, Jul. 2023, pp. 1762–1777.
- [25] Z. Wang, A. Liu, H. Lin, J. Li, X. Ma, and Y. Liang, "RAT: Retrieval augmented thoughts elicit context-aware reasoning in long-horizon generation," *arXiv:2403.05313*, 2024.
- [26] S. V. Hoeyweghen, "Speaking of games: AI-based content moderation of real-time voice interactions in video games under the DSA," *Interactive Entertainment Law Review*, vol. 7, no. 1, pp. 30–46, 2024.
- [27] B. Y. Lin, Y. Fu, K. Yang, F. Brahman, S. Huang, C. Bhagavatula, P. Ammanabrolu, Y. Choi, and X. Ren, "SwiftSage: A generative agent with fast and slow thinking for complex interactive tasks," in *Advances in Neural Information Processing Systems*, New Orleans, United States, Dec. 2023, pp. 23 813–23 825.
- [28] V. Pallagani, B. C. Muppasani, K. Roy, F. Fabiano, A. Loreggia, K. Murugesan, S. Srivastava, F. Rossi, L. Horeh, and A. Sheth, "On the prospects of incorporating large language models in automated planning and scheduling," in *Proceedings of the International Conference on Automated Planning and Scheduling*, Banaff, Alberta, Canada, May 2024, pp. 432–444.
- [29] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems*, New Orleans, United States, Nov. 2022, pp. 24 824–24 837.
- [30] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafra, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proceedings of the International Conference on Learning Representations*, Apr. 2022, pp. 1–33.
- [31] Y. Shen, K. Song, X. Tan, D. Li, W. Lu, and Y. Zhuang, "HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face," in *Advances in Neural Information Processing Systems*, New Orleans, United States, Dec. 2023, pp. 38 154–38 180.
- [32] J. Long, "Large language model guided tree-of-thought," *arXiv:2305.08291*, 2023.
- [33] M. Besta, N. Blach, A. Kubicek, R. Gerstenberger, M. Podstawski, L. Gianinazzi, J. Gajda, T. Lehmann, H. Niewiadomski, P. Nycz, and T. Hoefler, "Graph of thoughts: Solving elaborate problems with large language models," in *Proceedings of the AAAI Conference on Artificial Intelligence*. Vancouver, Canada: AAAI, Mar. 2024, pp. 17 682–17 690.
- [34] W. Fan, Y. Ding, L. Ning, S. Wang, H. Li, D. Yin, T.-S. Chua, and Q. Li, "A survey on RAG meeting LLMs: Towards retrieval-augmented large language models," in *Proceedings of the ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, New York, USA, Aug. 2024, pp. 6491–6501.
- [35] W. Peng, G. Li, Y. Jiang, Z. Wang, D. Ou, X. Zeng, D. Xu, T. Xu, and E. Chen, "Large language model based long-tail query rewriting in taobao search," in *Companion Proceedings of the ACM Web Conference*, New York, USA, May 2024, pp. 20–28.
- [36] X. Li and J. Li, "AoE: Angle-optimized embeddings for semantic textual similarity," in *Proceedings of the Annual Meeting of the Association for Computational Linguistics*, Bangkok, Thailand, Aug. 2024, pp. 1825–1839.
- [37] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," *Transactions on Machine Learning Research*, vol. 3, pp. 1–44, 2024.
- [38] D. Kahneman, *Thinking, Fast and Slow*. Macmillan, 2011.
- [39] S. M. Xie, A. Raghunathan, P. Liang, and T. Ma, "An explanation of in-context learning as implicit bayesian inference," in *International Conference on Learning Representations*, Virtual, Apr. 2022, pp. 1–25.
- [40] R. Hendel, M. Geva, and A. Globerson, "In-context learning creates task vectors," in *Proceedings of the Conference on Empirical Methods in Natural Language Processing*, Singapore, Dec. 2023, pp. 1–16.
- [41] C. Chan, W. Chen, Y. Su, J. Yu, W. Xue, S. Zhang, J. Fu, and Z. Liu, "ChatEval: Towards better LLM-based evaluators through multi-agent debate," in *Proceedings of the International Conference on Learning Representations*, Vienna, Austria, May 2024, pp. 1–15.
- [42] S. Peleg and B. Porat, "The Cramer-Rao lower bound for signals with constant amplitude and polynomial phase," *IEEE Transactions on Signal Processing*, vol. 39, no. 3, pp. 749–752, 1991.
- [43] E. Dale, *Audio-Visual Methods in Teaching*. New York: Dryden Press, 1996.
- [44] W. Zeng, K. He, Y. Wang, C. Zeng, G. Wang, Y. Xian, and W. Xu, "FutureTOD: Teaching future knowledge to pre-trained language model for task-oriented dialogue," in *Proceedings of the Association for Computational Linguistics*. Toronto, Canada: ACL, Jul. 2023, pp. 6532–6546.
- [45] G. Luo. Pokemon-Showdown. [Online]. Available: <https://play.pokemonshowdown.com>
- [46] H. Sahovic. Poke-env: Pokemon AI in python. [Online]. Available: <https://github.com/hsahovic/poke-env>
- [47] S. Karden, A. L. Nguyen, and C. Jin, "PokeChamp: an expert-level minimax language agent for competitive pokemon," *arXiv:2503.04094*, 2025.
- [48] J. Grigsby, Y. Xie, J. Sasek, S. Zheng, and Y. Zhu, "Human-level competitive pokemon via scalable offline reinforcement learning with transformers," *arXiv:2504.04395*, 2025.
- [49] D. Fu, J. Huang, S. Lu, G. Dong, Y. Wang, K. He, and W. Xu, "PreAct: Predicting future in ReAct enhances agent's planning ability," *arXiv:2402.11534*, 2024.
- [50] D. M. Monetti, J. H. Hummel, and W. G. Huitt, "Educational psychology principles that contribute to effective teaching and learning," *International Journal of Arts & Sciences*, vol. 1, no. 1, pp. 22–25, 2006.
- [51] R. S. Reldman, *Understanding Psychology*. McGrawHill, 2015.
- [52] M. Prensky, *Digital Game-Based Learning*. McGrawHill, 2001.
- [53] A. C. Siang and R. K. Rao, "Theories of learning: a computer game perspective," in *International Symposium on Multimedia Software Engineering*, Taiwan, China, Dec. 2003, pp. 239–245.
- [54] Introducing Meta Llama 3: The most capable openly available LLM to date. [Online]. Available: <https://ai.meta.com/blog/meta-llama-3/>
- [55] T. H. Morris, "Experiential learning - a systematic review and revision of Kolb's model," *Interactive Learning Environments*, vol. 28, no. 8, pp. 1064–1077, 2020.
- [56] Heartharena. [Online]. Available: <https://go.overwolf.com/heartharena/>
- [57] D. Moshman, "Metacognitive theories revisited," *Educational Psychology Review*, vol. 30, pp. 599–606, 2018.